

crash



crash

- *crash* is a CLI tool allowing to investigate kernel (dead or alive!)
  - Uses /dev/mem or /proc/kcore on live systems
  - Requires CONFIG_STRICT_DEVMEM=n
- Can use a coredump generated using kdump, kvmdump, etc.
- Based on `gdb` and provides many specific commands to inspect the kernel state.
  - Stack traces, dmesg (`log`), memory maps of the processes, irqs, virtual memory areas, etc.
- Allows examining all the tasks that are running on the system.
- Hosted at https://github.com/crash-utility/crash

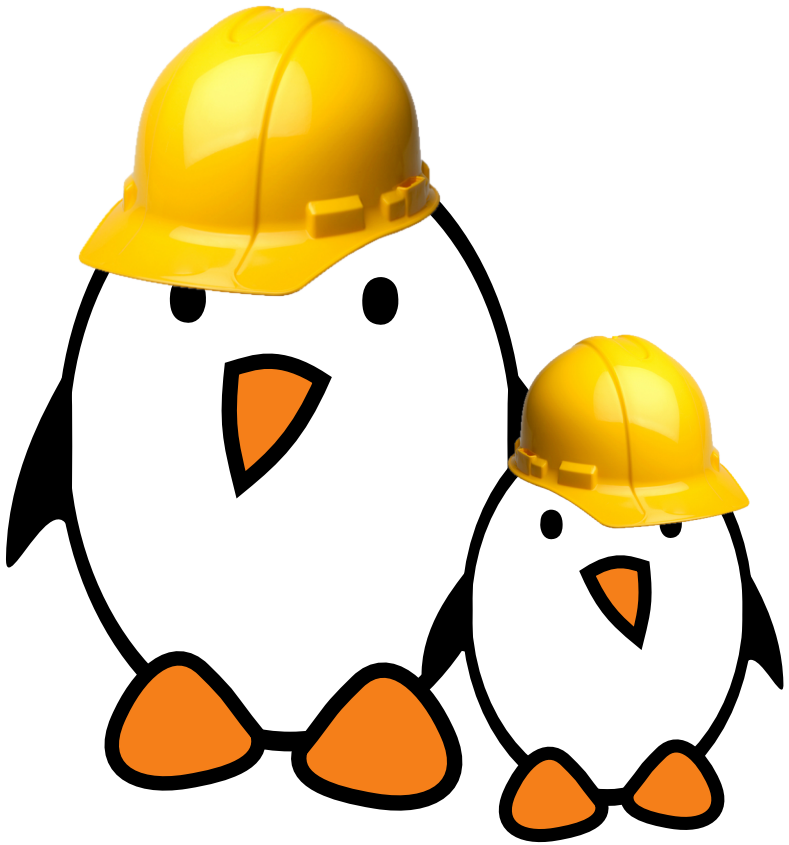


crash example

```
$ crash vmlinux vmcore
[...]
    TASKS: 75
NODENAME: buildroot
  RELEASE: 5.13.0
  VERSION: #1 SMP PREEMPT Tue Nov 15 14:42:25 CET 2022
  MACHINE: armv7l  (unknown Mhz)
  MEMORY: 512 MB
    PANIC: "Unable to handle kernel NULL pointer dereference at virtual address 00000070"
      PID: 127
  COMMAND: "watchdog"
    TASK: c3f163c0  [THREAD_INFO: c3f00000]
      CPU: 1
    STATE: TASK_RUNNING (PANIC)

crash> mach
    MACHINE TYPE: armv7l
     MEMORY SIZE: 512 MB
            CPUS: 1
 PROCESSOR SPEED: (unknown)
              HZ: 100
       PAGE SIZE: 4096
KERNEL VIRTUAL BASE: c0000000
KERNEL MODULES BASE: bf000000
KERNEL VMALLOC BASE: e0000000
KERNEL STACK SIZE: 8192
```

Practical lab - Kernel debugging



Debugging kernel crashes on a live kernel

- Analyze an OOPS message
- Debug a crash with KGDB



Post-mortem analysis



Kernel crash post-mortem analysis

- Sometimes, accessing the crashed system is not possible or the system can't stay offline while waiting to be debugged
- Kernel can generate crash dumps (a *vmcore* file) to a remote location, allowing to quickly restart the system while still be able to perform post-mortem analysis with GDB.
- This feature relies on *kexec* and *kdump* which will boot another kernel as soon as the crash occurs right after dumping the *vmcore* file.
  - The *vmcore* file can be saved on local storage, via SSH, FTP etc.



kexec & kdump (1/2)

- On panic, the kernel kexec support will execute a “dump-capture kernel” directly from the kernel that crashed
  - Most of the time, a specific dump-capture kernel is compiled for that task (minimal config with specific initramfs/initrd)
- *kexec* system works by saving some RAM for the kdump kernel execution at startup
  - `crashkernel` parameter should be set to specify the crash kernel dedicated physical memory region
- *kexec-tools* are then used to load dump-capture kernel into this memory zone using the `kexec` command
  - Internally uses the `kexec_load` system call man 2 kexec_load



kexec & kdump (2/2)

- Finally, on panic, the kernel will reboot into the "dump-capture" kernel allowing the user to dump the kernel coredump (`/proc/vmcore`) onto whatever media
- Additional command line options depends on the architecture
- See admin-guide/kdump/kdump for more comprehensive explanations on how to setup the kdump kernel with `kexec`.
- Additional user-space services and tools allow to automatically collect and dump the vmcore file to a remote location.
  - See kdump systemd service and the `makedumpfile` tool which can also compress the vmcore file into a smaller file (Only for x86, PPC, IA64, S390).
  - https://github.com/makedumpfile/makedumpfile

kdump

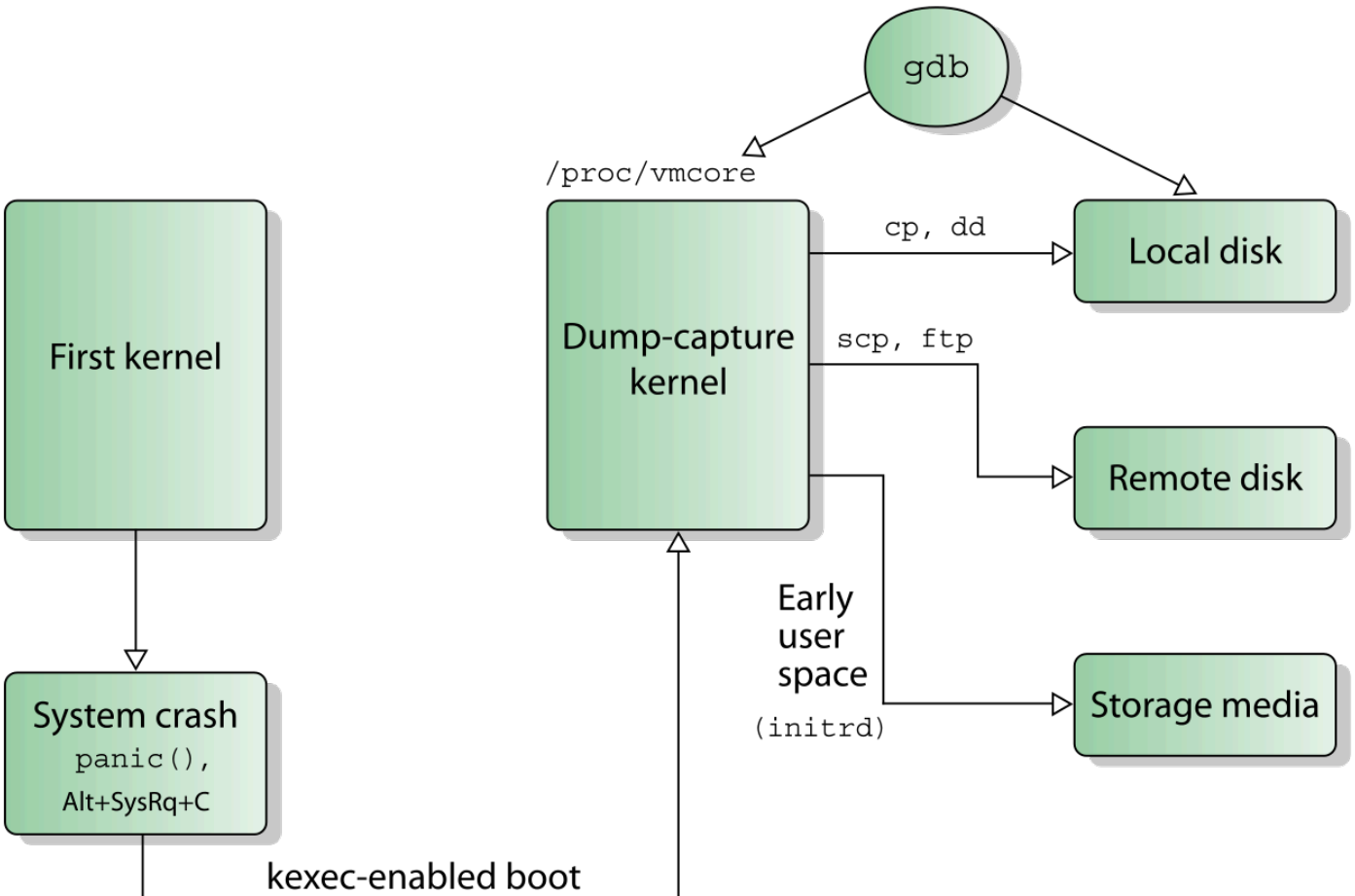


Image credits: Wikipedia



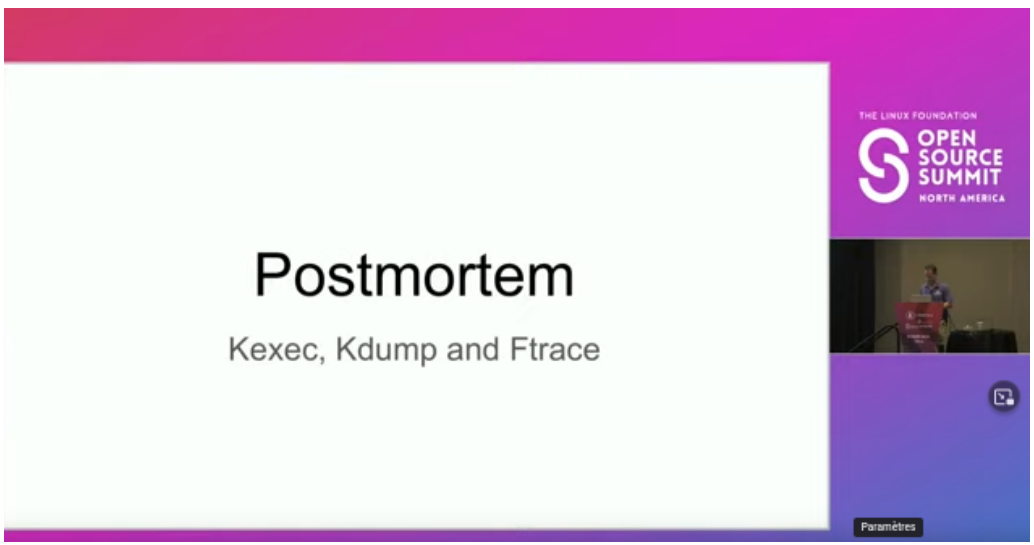
kexec config and setup

- On the standard kernel:
  - CONFIG_KEXEC=y to enable KEXEC support
  - `kexec-tools` to provide the `kexec` command
  - A kernel and a DTB accessible by `kexec`
- On the dump-capture kernel:
  - CONFIG_CRASH_DUMP=y to enable dumping a crashed kernel
  - CONFIG_PROC_VMCORE=y to enable `/proc/vmcore` support
  - CONFIG_AUTO_ZRELADDR=y on ARM32 platforms
- Set the correct `crashkernel` command line option:
  - `crashkernel=size[KMG][@offset[KMG]]`
- Load a dump-capture kernel on the first kernel with `kexec`:
  - ```
    kexec --type zImage -p my_zImage --dtb=my_dtb.dtb \
    --initrd=my_initrd --command-line="kernel command line"
    ```
- Then simply wait for a crash to happen!
    ```



Going further with kexec & kdump

- Presentation from Steven Rostedt about using kexec, kdump and ftrace with lot of tips and tricks about using kexec/kdump
  - Video: https://www.youtube.com/watch?v=aUGNDJPpUUg
  - Slides: https://static.sched.com/hosted_files/ossna2022/c0/Postmortem_%20Kexec%2C%20Kdump%20and%20Ftrace.pdf





pstore (1/3)

- Linux provides a filesystem interface for Persistent Storage (`pstore`) to save data across system resets: kernel logs, oopses, ftrace records, user messages...
- The platform needs to provide a persistent area to pstore (a block device, reserved RAM which is not reset on reboot, etc). Then you can enable a pstore frontend.
- ramoops is a common frontend for pstore: it will log any panic/oops to a pstore-managed ram buffer, which will be accessible on next boot
- Saved logs can be retrieved on next boot thanks to the pstore filesystem
- Some earlier software components in the boot chain (eg: U-Boot), if properly configured, may be able to access pstore data as well



pstore (2/3)

- Kernel configuration:
  - CONFIG_PSTORE=y
  - CONFIG_PSTORE_RAM=y
- Platform configuration: reserve some memory for pstore and configure it
  - Either through kernel command line:

```
mem=<usable_memory_size> ramoops.mem_address=0x8000000 ramoops.ecc=1
```

  - Or through device tree:

```
reserved-memory {
        [...]
        ramoops@8f000000 {
                compatible = "ramoops";
                reg = <0 0x8f000000 0 0x100000>;
                record-size = <0x4000>;
                console-size = <0x4000>;
        };
};
```



pstore (3/3)

- After a crash, the collected logs/traces will be available in the pstore filesystem:

```
mount -t pstore pstore /sys/fs/pstore
```

- If your data is not present in pstore filesystem, some services spawned early may have already collected/moved it
  - For example: systemd-pstore

Practical lab - Kernel debugging



Post-mortem debugging of a kernel crash

- Setup kexec, kdump and extract a kernel coredump

Going further



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



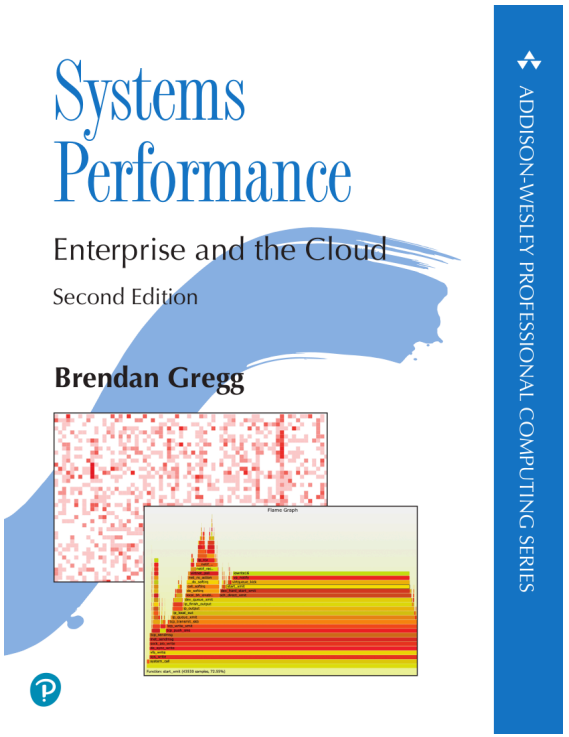
Debugging resources

- Brendan Gregg Systems performance book
- Brendan Gregg Linux Performance page
- *Tools and Techniques to Debug an Embedded Linux System*, talk from Sergio Prado, video, slides
- *Tracing with Ftrace: Critical Tooling for Linux Development*, talk from Steven Rostedt, video
- *Tutorial: Debugging Embedded Devices using GDB*, tutorial from Chris Simmonds, video



Going further (Tracing & Profiling)

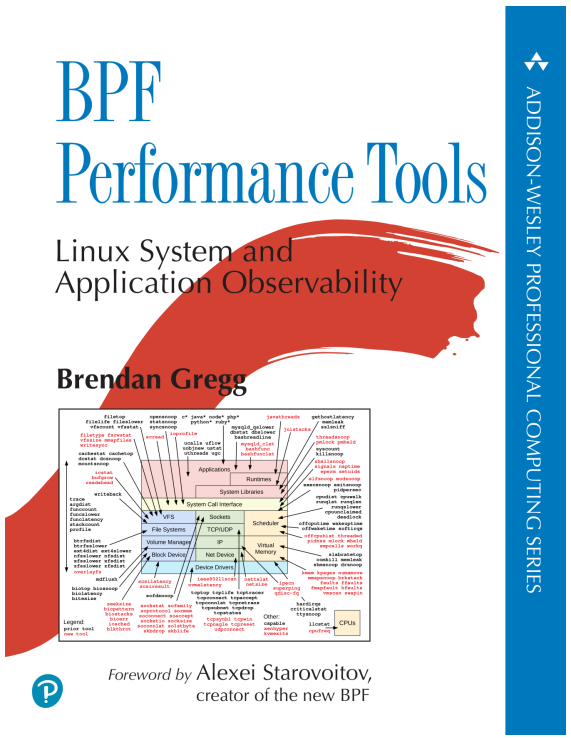
- Great book from Brendan Gregg, an expert in tracing and profiling
- https://www.brendangregg.com/blog/2020-07-15/systems-performance-2nd-edition.html
- Covers concepts, strategy, tools, and tuning for Linux kernel and applications.





Going further (BPF)

- Still from Brendan Gregg!
- Covers more than 150 tools that use BPF.
- Explains how to analyze the results from these tools to optimize your system.
- https://www.brendangregg.com/bpf-performance-tools-book.html



Last slides

© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!





Last slide

Thank you!

And may the Source be with you



Rights to copy

© Copyright 2004-2026, Bootlin
License: Creative Commons Attribution - Share Alike 3.0
https://creativecommons.org/licenses/by-sa/3.0/legalcode
You are free:

- to copy, distribute, display, and perform the work
- to make derivative works
- to make commercial use of the work

Under the following conditions:

- **Attribution**. You must give the original author credit.
- **Share Alike**. If you alter, transform, or build upon this work, you may distribute the resulting work only under a license identical to this one.
- For any reuse or distribution, you must make clear to others the license terms of this work.
- Any of these conditions can be waived if you get permission from the copyright holder.

Your fair use and other rights are in no way affected by the above.

Document sources: https://github.com/bootlin/training-materials/